



JS

Day 6

Network requests

FormData

Fetch: Cross-Origin Requests

URL objects

WebSocket

Network requests

JavaScript can send network requests to the server and load new information whenever it's needed.

For example, we can use a network request to:

- Submit an order,
- Load user information,
- Receive latest updates from the server,
- ...etc.

...And all of that without reloading the page!

There's an umbrella term "AJAX" (abbreviated **A**synchronous **J**avaScript **A**nd **X**ML) for network requests from JavaScript. We don't have to use XML though: the term comes from old times, that's why that word is there. You may have heard that term already.

There are multiple ways to send a network request and get information from the server.

FormData

This chapter is about sending HTML forms: with or without files, with additional fields and so on.

`FormData` [↗](#) objects can help with that. As you might have guessed, it's the object to represent HTML form data.

The constructor is:

```
let formData = new FormData([form]);
```

If HTML `form` element is provided, it automatically captures its fields.

The special thing about `FormData` is that network methods, such as `fetch`, can accept a `FormData` object as a body. It's encoded and sent out with `Content-Type: multipart/form-data`.

From the server point of view, that looks like a usual form submission.

Sending a simple form

Let's send a simple form first.

As you can see, that's almost one-liner:

```
<form id="formElem">
  <input type="text" name="name" value="John">
  <input type="text" name="surname" value="Smith">
  <input type="submit">
</form>

<script>
  formElem.onsubmit = async (e) => {
    e.preventDefault();

    let response = await fetch('/article/formdata/post/user', {
      method: 'POST',
      body: new FormData(formElem)
    });

    let result = await response.json();

    alert(result.message);
  };
</script>
```

In this example, the server code is not presented, as it's beyond our scope. The server accepts the POST request and replies "User saved".

FormData Methods

We can modify fields in `FormData` with methods:

- `formData.append(name, value)` – add a form field with the given `name` and `value`,
- `formData.append(name, blob, fileName)` – add a field as if it were `<input type="file">`, the third argument `fileName` sets file name (not form field name), as it were a name of the file in user's filesystem,
- `formData.delete(name)` – remove the field with the given `name`,
- `formData.get(name)` – get the value of the field with the given `name`,
- `formData.has(name)` – if there exists a field with the given `name`, returns `true`, otherwise `false`

A form is technically allowed to have many fields with the same `name`, so multiple calls to `append` add more same-named fields.

There's also method `set`, with the same syntax as `append`. The difference is that `.set` removes all fields with the given `name`, and then appends a new field. So it

makes sure there's only one field with such `name`, the rest is just like `append`:

- `formData.set(name, value),`
- `formData.set(name, blob, fileName).`

Also we can iterate over `formData` fields using `for...of` loop:

```
let formData = new FormData();
formData.append('key1', 'value1');
formData.append('key2', 'value2');

// List key/value pairs
for(let [name, value] of formData) {
  alert(`${name} = ${value}`); // key1 = value1, then key2 = value2
}
```

Sending a form with a file

The form is always sent as `Content-Type: multipart/form-data`, this encoding allows to send files. So, `<input type="file">` fields are sent also, similar to a usual form submission.

Here's an example with such form:

```
<form id="formElem">
  <input type="text" name="firstName" value="John">
  Picture: <input type="file" name="picture" accept="image/*">
  <input type="submit">
</form>

<script>
  formElem.onsubmit = async (e) => {
    e.preventDefault();

    let response = await fetch('/article/formdata/post/user-avatar', {
      method: 'POST',
      body: new FormData(formElem)
    });

    let result = await response.json();

    alert(result.message);
  };
</script>
```

Picture: No file chosen

Sending a form with Blob data

As we've seen in the chapter [Fetch](#), it's easy to send dynamically generated binary data e.g. an image, as `Blob`. We can supply it directly as `fetch` parameter `body`.

In practice though, it's often convenient to send an image not separately, but as a part of the form, with additional fields, such as "name" and other metadata.

Also, servers are usually more suited to accept multipart-encoded forms, rather than raw binary data.

This example submits an image from `<canvas>`, along with some other fields, as a form, using `FormData`:

```
<body style="margin:0">
  <canvas id="canvasElem" width="100" height="80" style="border:1px solid"></c

  <input type="button" value="Submit" onclick="submit()">

  <script>
    canvasElem.onmousemove = function(e) {
      let ctx = canvasElem.getContext('2d');
      ctx.lineTo(e.clientX, e.clientY);
      ctx.stroke();
    };

    async function submit() {
      let imageBlob = await new Promise(resolve => canvasElem.toBlob(resolve,

      let formData = new FormData();
      formData.append("firstName", "John");
      formData.append("image", imageBlob, "image.png");

      let response = await fetch('/article/formdata/post/image-form', {
        method: 'POST',
        body: formData
      });
      let result = await response.json();
      alert(result.message);
    }

  </script>
</body>
```

Submit

```
formData.append("image", imageBlob, "image.png");
```

That's same as if there were `<input type="file" name="image">` in the form, and the visitor submitted a file named `"image.png"` (3rd argument) with the data `imageBlob` (2nd argument) from their filesystem.

The server reads form data and the file, as if it were a regular form submission.

Summary

[FormData](#) [↗](#) objects are used to capture HTML form and submit it using `fetch` or another network method.

We can either create `new FormData(form)` from an HTML form, or create an object without a form at all, and then append fields with methods:

- `formData.append(name, value)`
- `formData.append(name, blob, fileName)`
- `formData.set(name, value)`
- `formData.set(name, blob, fileName)`

Let's note two peculiarities here:

1. The `set` method removes fields with the same name, `append` doesn't. That's the only difference between them.
2. To send a file, 3-argument syntax is needed, the last argument is a file name, that normally is taken from user filesystem for `<input type="file">`.

Other methods are:

- `formData.delete(name)`
- `formData.get(name)`
- `formData.has(name)`

Fetch: Cross-Origin Requests

If we send a `fetch` request to another web-site, it will probably fail.

For instance, let's try fetching `http://example.com`:

```
try {
  await fetch('http://example.com');
} catch(err) {
  alert(err); // Failed to fetch
}
```

Fetch fails, as expected.

The core concept here is *origin* – a domain/port/protocol triplet.

Cross-origin requests – those sent to another domain (even a subdomain) or protocol or port – require special headers from the remote side.

That policy is called “CORS”: Cross-Origin Resource Sharing.

Why is CORS needed? A brief history

CORS exists to protect the internet from evil hackers.

Seriously. Let's make a very brief historical digression.

For many years a script from one site could not access the content of another site.

That simple, yet powerful rule was a foundation of the internet security. E.g. an evil script from website `hacker.com` could not access the user's mailbox at website `gmail.com`. People felt safe.

JavaScript also did not have any special methods to perform network requests at that time. It was a toy language to decorate a web page.

But web developers demanded more power. A variety of tricks were invented to work around the limitation and make requests to other websites.

Using forms

One way to communicate with another server was to submit a `<form>` there. People submitted it into `<iframe>`, just to stay on the current page, like this:

```
<!-- form target -->
<iframe name="iframe"></iframe>

<!-- a form could be dynamically generated and submitted by JavaScript -->
<form target="iframe" method="POST" action="http://another.com/...">
```

```
...  
</form>
```

So, it was possible to make a GET/POST request to another site, even without networking methods, as forms can send data anywhere. But as it's forbidden to access the content of an `<iframe>` from another site, it wasn't possible to read the response.

To be precise, there were actually tricks for that, they required special scripts at both the iframe and the page. So the communication with the iframe was technically possible. Right now there's no point to go into details, let these dinosaurs rest in peace.

Using scripts

Another trick was to use a `script` tag. A script could have any `src`, with any domain, like `<script src="http://another.com/...">`. It's possible to execute a script from any website.

If a website, e.g. `another.com` intended to expose data for this kind of access, then a so-called "JSONP (JSON with padding)" protocol was used.

Here's how it worked.

Let's say we, at our site, need to get the data from `http://another.com`, such as the weather:

1. First, in advance, we declare a global function to accept the data, e.g. `gotWeather`.

```
// 1. Declare the function to process the weather data  
function gotWeather({ temperature, humidity }) {  
  alert(`temperature: ${temperature}, humidity: ${humidity}`);  
}
```

2. Then we make a `<script>` tag with `src="http://another.com/weather.json?callback=gotWeather"`, using the name of our function as the `callback` URL-parameter.

```
let script = document.createElement('script');  
script.src = `http://another.com/weather.json?callback=gotWeather`;  
document.body.append(script);
```

3. The remote server `another.com` dynamically generates a script that calls `gotWeather(...)` with the data it wants us to receive.

```
// The expected answer from the server looks like this:  
gotWeather({  
  temperature: 25,  
  humidity: 78  
});
```

4. When the remote script loads and executes, `gotWeather` runs, and, as it's our function, we have the data.

That works, and doesn't violate security, because both sides agreed to pass the data this way. And, when both sides agree, it's definitely not a hack. There are still services that provide such access, as it works even for very old browsers.

After a while, networking methods appeared in browser JavaScript.

At first, cross-origin requests were forbidden. But as a result of long discussions, cross-origin requests were allowed, but with any new capabilities requiring an explicit allowance by the server, expressed in special headers.

Safe requests

There are two types of cross-origin requests:

1. Safe requests.
2. All the others.

Safe Requests are simpler to make, so let's start with them.

A request is safe if it satisfies two conditions:

1. [Safe method](#) : GET, POST or HEAD
2. [Safe headers](#) – the only allowed custom headers are:
 - `Accept` ,
 - `Accept-Language` ,
 - `Content-Language` ,
 - `Content-Type` with the value `application/x-www-form-urlencoded` , `multipart/form-data` or `text/plain` .

Any other request is considered “unsafe”. For instance, a request with `PUT` method or with an `API-Key` HTTP-header does not fit the limitations.

The essential difference is that a safe request can be made with a `<form>` or a `<script>` , without any special methods.

So, even a very old server should be ready to accept a safe request.

Contrary to that, requests with non-standard headers or e.g. method `DELETE` can't be created this way. For a long time JavaScript was unable to do such requests. So an old server may assume that such requests come from a privileged source, "because a webpage is unable to send them".

When we try to make a unsafe request, the browser sends a special "preflight" request that asks the server – does it agree to accept such cross-origin requests, or not?

And, unless the server explicitly confirms that with headers, an unsafe request is not sent.

Now we'll go into details.

CORS for safe requests

If a request is cross-origin, the browser always adds the `Origin` header to it.

For instance, if we request `https://anywhere.com/request` from `https://javascript.info/page`, the headers will look like:

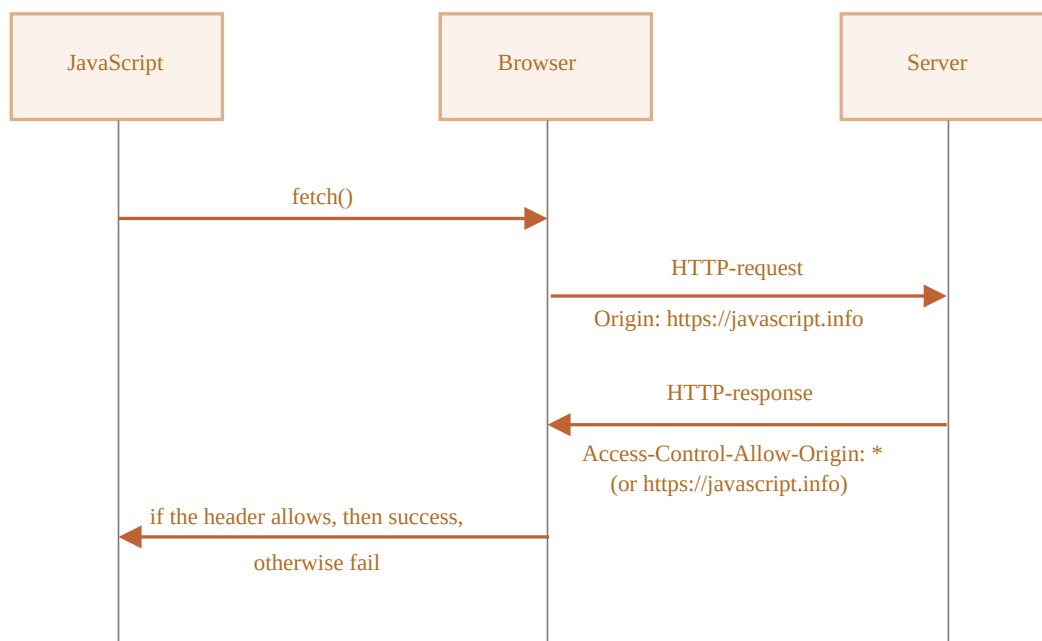
```
GET /request
Host: anywhere.com
Origin: https://javascript.info
...
```

As you can see, the `Origin` header contains exactly the origin (domain/protocol/port), without a path.

The server can inspect the `Origin` and, if it agrees to accept such a request, add a special header `Access-Control-Allow-Origin` to the response. That header should contain the allowed origin (in our case `https://javascript.info`), or a star `*`. Then the response is successful, otherwise it's an error.

The browser plays the role of a trusted mediator here:

1. It ensures that the correct `Origin` is sent with a cross-origin request.
2. It checks for permitting `Access-Control-Allow-Origin` in the response, if it exists, then JavaScript is allowed to access the response, otherwise it fails with an error.



Here's an example of a permissive server response:

```
200 OK
Content-Type: text/html; charset=UTF-8
Access-Control-Allow-Origin: https://javascript.info
```

Response headers

For cross-origin request, by default JavaScript may only access so-called “safe” response headers:

- Cache-Control
- Content-Language
- Content-Length
- Content-Type
- Expires
- Last-Modified
- Pragma

Accessing any other response header causes an error.

To grant JavaScript access to any other response header, the server must send the `Access-Control-Expose-Headers` header. It contains a comma-separated list of unsafe header names that should be made accessible.

For example:

```
200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 12345
Content-Encoding: gzip
API-Key: 2c9de507f2c54aa1
Access-Control-Allow-Origin: https://javascript.info
Access-Control-Expose-Headers: Content-Encoding, API-Key
```

With such an `Access-Control-Expose-Headers` header, the script is allowed to read the `Content-Encoding` and `API-Key` headers of the response.

“Unsafe” requests

We can use any HTTP-method: not just `GET/POST`, but also `PATCH`, `DELETE` and others.

Some time ago no one could even imagine that a webpage could make such requests. So there may still exist webservices that treat a non-standard method as a signal: “That’s not a browser”. They can take it into account when checking access rights.

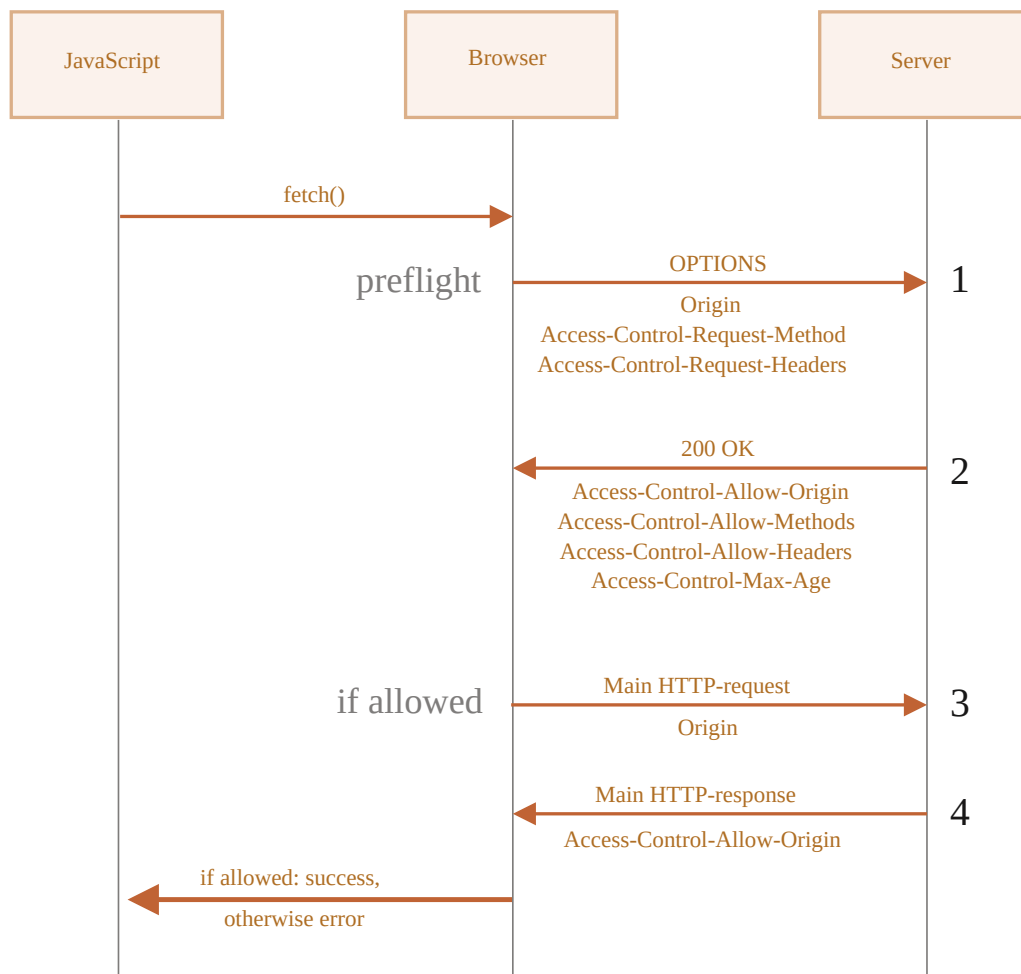
So, to avoid misunderstandings, any “unsafe” request – that couldn’t be done in the old times, the browser does not make such requests right away. First, it sends a preliminary, so-called “preflight” request, to ask for permission.

A preflight request uses the method `OPTIONS`, no body and three headers:

- `Access-Control-Request-Method` header has the method of the unsafe request.
- `Access-Control-Request-Headers` header provides a comma-separated list of its unsafe HTTP-headers.
- `Origin` header tells from where the request came. (such as `https://javascript.info`)

If the server agrees to serve the requests, then it should respond with empty body, status 200 and headers:

- `Access-Control-Allow-Origin` must be either `*` or the requesting origin, such as `https://javascript.info`, to allow it.
- `Access-Control-Allow-Methods` must have the allowed method.
- `Access-Control-Allow-Headers` must have a list of allowed headers.
- Additionally, the header `Access-Control-Max-Age` may specify a number of seconds to cache the permissions. So the browser won’t have to send a preflight for subsequent requests that satisfy given permissions.



Let's see how it works step-by-step on the example of a cross-origin `PATCH` request (this method is often used to update data):

```

let response = await fetch('https://site.com/service.json', {
  method: 'PATCH',
  headers: {
    'Content-Type': 'application/json',
    'API-Key': 'secret'
  }
});

```

There are three reasons why the request is unsafe (one is enough):

- Method `PATCH`
- `Content-Type` is not one of: `application/x-www-form-urlencoded`, `multipart/form-data`, `text/plain`.
- “Unsafe” `API-Key` header.

Step 1 (preflight request)

Prior to sending such a request, the browser, on its own, sends a preflight request that looks like this:

```
OPTIONS /service.json
Host: site.com
Origin: https://javascript.info
Access-Control-Request-Method: PATCH
Access-Control-Request-Headers: Content-Type,API-Key
```

- Method: `OPTIONS`.
- The path – exactly the same as the main request: `/service.json`.
- Cross-origin special headers:
 - `Origin` – the source origin.
 - `Access-Control-Request-Method` – requested method.
 - `Access-Control-Request-Headers` – a comma-separated list of “unsafe” headers.

Step 2 (preflight response)

The server should respond with status 200 and the headers:

- `Access-Control-Allow-Origin: https://javascript.info`
- `Access-Control-Allow-Methods: PATCH`
- `Access-Control-Allow-Headers: Content-Type,API-Key`.

That allows future communication, otherwise an error is triggered.

If the server expects other methods and headers in the future, it makes sense to allow them in advance by adding them to the list.

For example, this response also allows `PUT`, `DELETE` and additional headers:

```
200 OK
Access-Control-Allow-Origin: https://javascript.info
Access-Control-Allow-Methods: PUT,PATCH,DELETE
Access-Control-Allow-Headers: API-Key,Content-Type,If-Modified-Since,Cache-Con
Access-Control-Max-Age: 86400
```

Now the browser can see that `PATCH` is in `Access-Control-Allow-Methods` and `Content-Type,API-Key` are in the list `Access-Control-Allow-Headers`, so it sends out the main request.

If there's the header `Access-Control-Max-Age` with a number of seconds, then the preflight permissions are cached for the given time. The response above will be

cached for 86400 seconds (one day). Within this timeframe, subsequent requests will not cause a preflight. Assuming that they fit the cached allowances, they will be sent directly.

Step 3 (actual request)

When the preflight is successful, the browser now makes the main request. The process here is the same as for safe requests.

The main request has the `Origin` header (because it's cross-origin):

```
PATCH /service.json
Host: site.com
Content-Type: application/json
API-Key: secret
Origin: https://javascript.info
```

Step 4 (actual response)

The server should not forget to add `Access-Control-Allow-Origin` to the main response. A successful preflight does not relieve from that:

```
Access-Control-Allow-Origin: https://javascript.info
```

Then JavaScript is able to read the main server response.

Please note:

Preflight request occurs “behind the scenes”, it's invisible to JavaScript.

JavaScript only gets the response to the main request or an error if there's no server permission.

Credentials

A cross-origin request initiated by JavaScript code by default does not bring any credentials (cookies or HTTP authentication).

That's uncommon for HTTP-requests. Usually, a request to `http://site.com` is accompanied by all cookies from that domain. Cross-origin requests made by JavaScript methods on the other hand are an exception.

For example, `fetch('http://another.com')` does not send any cookies, even those (!) that belong to `another.com` domain.

Why?

That's because a request with credentials is much more powerful than without them. If allowed, it grants JavaScript the full power to act on behalf of the user and access sensitive information using their credentials.

Does the server really trust the script that much? Then it must explicitly allow requests with credentials with an additional header.

To send credentials in `fetch`, we need to add the option `credentials: "include"`, like this:

```
fetch('http://another.com', {  
  credentials: "include"  
});
```

Now `fetch` sends cookies originating from `another.com` with request to that site.

If the server agrees to accept the request *with credentials*, it should add a header `Access-Control-Allow-Credentials: true` to the response, in addition to `Access-Control-Allow-Origin`.

For example:

```
200 OK  
Access-Control-Allow-Origin: https://javascript.info  
Access-Control-Allow-Credentials: true
```

Please note: `Access-Control-Allow-Origin` is prohibited from using a star `*` for requests with credentials. Like shown above, it must provide the exact origin there. That's an additional safety measure, to ensure that the server really knows who it trusts to make such requests.

Summary

From the browser point of view, there are two kinds of cross-origin requests: "safe" and all the others.

"Safe" requests must satisfy the following conditions:

- Method: GET, POST or HEAD.
- Headers – we can set only:
 - `Accept`
 - `Accept-Language`
 - `Content-Language`

- `Content-Type` to the value `application/x-www-form-urlencoded`, `multipart/form-data` or `text/plain`.

The essential difference is that safe requests were doable since ancient times using `<form>` or `<script>` tags, while unsafe were impossible for browsers for a long time.

So, the practical difference is that safe requests are sent right away, with the `Origin` header, while for the other ones the browser makes a preliminary “preflight” request, asking for permission.

For safe requests:

- → The browser sends the `Origin` header with the origin.
- ← For requests without credentials (not sent by default), the server should set:
 - `Access-Control-Allow-Origin` to `*` or same value as `Origin`
- ← For requests with credentials, the server should set:
 - `Access-Control-Allow-Origin` to same value as `Origin`
 - `Access-Control-Allow-Credentials` to `true`

Additionally, to grant JavaScript access to any response headers except `Cache-Control`, `Content-Language`, `Content-Type`, `Expires`, `Last-Modified` or `Pragma`, the server should list the allowed ones in `Access-Control-Expose-Headers` header.

For unsafe requests, a preliminary “preflight” request is issued before the requested one:

- → The browser sends an `OPTIONS` request to the same URL, with the headers:
 - `Access-Control-Request-Method` has requested method.
 - `Access-Control-Request-Headers` lists unsafe requested headers.
- ← The server should respond with status 200 and the headers:
 - `Access-Control-Allow-Methods` with a list of allowed methods,
 - `Access-Control-Allow-Headers` with a list of allowed headers,
 - `Access-Control-Max-Age` with a number of seconds to cache the permissions.
- Then the actual request is sent, and the previous “safe” scheme is applied.

✓ Tasks

Why do we need Origin?

importance: 5

As you probably know, there's HTTP-header `Referer` , that usually contains an url of the page which initiated a network request.

For instance, when fetching `http://google.com` from `http://javascript.info/some/url` , the headers look like this:

```
Accept: */*
Accept-Charset: utf-8
Accept-Encoding: gzip,deflate,sdch
Connection: keep-alive
Host: google.com
Origin: http://javascript.info
Referer: http://javascript.info/some/url
```

As you can see, both `Referer` and `Origin` are present.

The questions:

1. Why `Origin` is needed, if `Referer` has even more information?
2. Is it possible that there's no `Referer` or `Origin` , or is it incorrect?

[To solution](#)

URL objects

The built-in `URL` [↗](#) class provides a convenient interface for creating and parsing URLs.

There are no networking methods that require exactly a `URL` object, strings are good enough. So technically we don't have to use `URL`. But sometimes it can be really helpful.

Creating a URL

The syntax to create a new `URL` object:

```
new URL(url, [base])
```

- `url` – the full URL or only path (if base is set, see below),
- `base` – an optional base URL: if set and `url` argument has only path, then the URL is generated relative to `base`.

For example:

```
let url = new URL('https://javascript.info/profile/admin');
```

These two URLs are same:

```
let url1 = new URL('https://javascript.info/profile/admin');
let url2 = new URL('/profile/admin', 'https://javascript.info');

alert(url1); // https://javascript.info/profile/admin
alert(url2); // https://javascript.info/profile/admin
```

We can easily create a new URL based on the path relative to an existing URL:

```
let url = new URL('https://javascript.info/profile/admin');
let newUrl = new URL('tester', url);

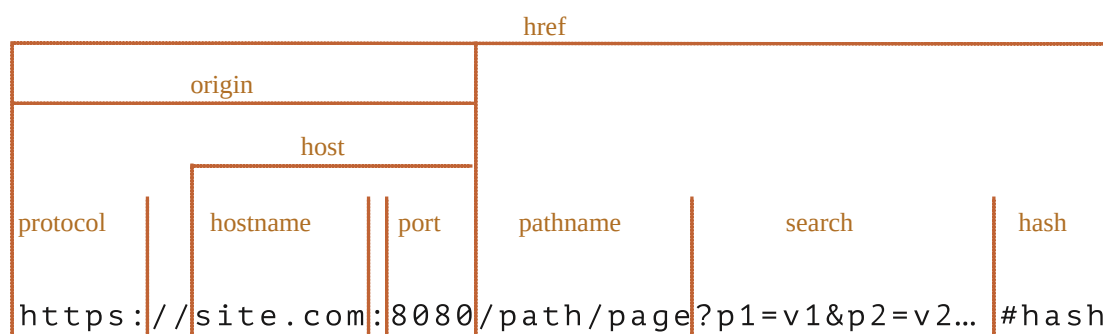
alert(newUrl); // https://javascript.info/profile/tester
```

The `URL` object immediately allows us to access its components, so it's a nice way to parse the url, e.g.:

```
let url = new URL('https://javascript.info/url');

alert(url.protocol); // https:
alert(url.host);      // javascript.info
alert(url.pathname); // /url
```

Here's the cheatsheet for URL components:



- `href` is the full url, same as `url.toString()`
- `protocol` ends with the colon character `:`
- `search` – a string of parameters, starts with the question mark `?`
- `hash` starts with the hash character `#`

- there may be also `user` and `password` properties if HTTP authentication is present: `http://login:password@site.com` (not painted above, rarely used).

i We can pass `URL` objects to networking (and most other) methods instead of a string

We can use a `URL` object in `fetch` or `XMLHttpRequest`, almost everywhere where a URL-string is expected.

Generally, the `URL` object can be passed to any method instead of a string, as most methods will perform the string conversion, that turns a `URL` object into a string with full URL.

SearchParams “?...”

Let's say we want to create a url with given search params, for instance, `https://google.com/search?query=JavaScript`.

We can provide them in the URL string:

```
new URL('https://google.com/search?query=JavaScript')
```

...But parameters need to be encoded if they contain spaces, non-latin letters, etc (more about that below).

So there's a URL property for that: `url.searchParams`, an object of type [URLSearchParams](#).

It provides convenient methods for search parameters:

- `append(name, value)` – add the parameter by `name`,
- `delete(name)` – remove the parameter by `name`,
- `get(name)` – get the parameter by `name`,
- `getAll(name)` – get all parameters with the same `name` (that's possible, e.g. `?user=John&user=Pete`),
- `has(name)` – check for the existence of the parameter by `name`,
- `set(name, value)` – set/replace the parameter,
- `sort()` – sort parameters by name, rarely needed,
- ...and it's also iterable, similar to `Map`.

An example with parameters that contain spaces and punctuation marks:

```

let url = new URL('https://google.com/search');

url.searchParams.set('q', 'test me!'); // added parameter with a space and !

alert(url); // https://google.com/search?q=test+me%21

url.searchParams.set('tbs', 'qdr:y'); // added parameter with a colon :

// parameters are automatically encoded
alert(url); // https://google.com/search?q=test+me%21&tbs=qdr%3Ay

// iterate over search parameters (decoded)
for(let [name, value] of url.searchParams) {
  alert(`${name}=${value}`); // q=test me!, then tbs=qdr:y
}

```

Encoding

There's a standard [RFC3986](#) that defines which characters are allowed in URLs and which are not.

Those that are not allowed, must be encoded, for instance non-latin letters and spaces – replaced with their UTF-8 codes, prefixed by %, such as %20 (a space can be encoded by +, for historical reasons, but that's an exception).

The good news is that `URL` objects handle all that automatically. We just supply all parameters unencoded, and then convert the `URL` to string:

```

// using some cyrillic characters for this example

let url = new URL('https://ru.wikipedia.org/wiki/Тест');

url.searchParams.set('key', 'б');
alert(url); //https://ru.wikipedia.org/wiki/%D0%A2%D0%B5%D1%81%D1%82?key=%D1%8

```

As you can see, both `Тест` in the url path and `б` in the parameter are encoded.

The URL became longer, because each cyrillic letter is represented with two bytes in UTF-8, so there are two %.. entities.

Encoding strings

In old times, before `URL` objects appeared, people used strings for URLs.

As of now, `URL` objects are often more convenient, but strings can still be used as well. In many cases using a string makes the code shorter.

If we use a string though, we need to encode/decode special characters manually.

There are built-in functions for that:

- [encodeURIComponent](#) – encodes URL as a whole.
- [decodeURI](#) – decodes it back.
- [encodeURIComponent](#) – encodes a URL component, such as a search parameter, or a hash, or a pathname.
- [decodeURIComponent](#) – decodes it back.

A natural question is: “What’s the difference between `encodeURIComponent` and `encodeURI` ? When we should use either?”

That’s easy to understand if we look at the URL, that’s split into components in the picture above:

```
https://site.com:8080/path/page?p1=v1&p2=v2#hash
```

As we can see, characters such as `:`, `?`, `=`, `&`, `#` are allowed in URL.

...On the other hand, if we look at a single URL component, such as a search parameter, these characters must be encoded, not to break the formatting.

- `encodeURI` encodes only characters that are totally forbidden in URL.
- `encodeURIComponent` encodes same characters, and, in addition to them, characters `#`, `$`, `&`, `+`, `,`, `/`, `:`, `;`, `=`, `?` and `@`.

So, for a whole URL we can use `encodeURI` :

```
// using cyrillic characters in url path
let url = encodeURI('http://site.com/привет');

alert(url); // http://site.com/%D0%BF%D1%80%D0%B8%D0%B2%D0%B5%D1%82
```

...While for URL parameters we should use `encodeURIComponent` instead:

```
let music = encodeURIComponent('Rock&Roll');

let url = `https://google.com/search?q=${music}`;
alert(url); // https://google.com/search?q=Rock%26Roll
```

Compare it with `encodeURI` :

```
let music = encodeURI('Rock&Roll');
```

```
let url = `https://google.com/search?q=${music}`;  
alert(url); // https://google.com/search?q=Rock&Roll
```

As we can see, `encodeURIComponent` does not encode `&`, as this is a legit character in URL as a whole.

But we should encode `&` inside a search parameter, otherwise, we get `q=Rock&Roll` – that is actually `q=Rock` plus some obscure parameter `Roll`. Not as intended.

So we should use only `encodeURIComponent` for each search parameter, to correctly insert it in the URL string. The safest is to encode both name and value, unless we're absolutely sure that it has only allowed characters.

i Encoding difference compared to URL

Classes [URL](#) and [URLSearchParams](#) are based on the latest URI specification: [RFC3986](#), while `encode*` functions are based on the obsolete version [RFC2396](#).

There are a few differences, e.g. IPv6 addresses are encoded differently:

```
// valid url with IPv6 address  
let url = 'http://[2607:f8b0:4005:802::1007]/';  
  
alert(encodeURIComponent(url)); // http://%5B2607:f8b0:4005:802::1007%5D/  
alert(new URL(url)); // http://[2607:f8b0:4005:802::1007]/
```

As we can see, `encodeURIComponent` replaced square brackets `[...]`, that's not correct, the reason is: IPv6 urls did not exist at the time of RFC2396 (August 1998).

Such cases are rare, `encode*` functions work well most of the time.

Some request methods like `GET` do not have a body. And some of them like `POST` use `body` to send the data to the server. We'll see examples of that later.

4. Listen to `xhr` events for response.

These three events are the most widely used:

- `load` – when the request is complete (even if HTTP status is like 400 or 500), and the response is fully downloaded.
- `error` – when the request couldn't be made, e.g. network down or invalid URL.
- `progress` – triggers periodically while the response is being downloaded, reports how much has been downloaded.

```
xhr.onload = function() {  
    alert(`Loaded: ${xhr.status} ${xhr.response}`);  
};  
  
xhr.onerror = function() { // only triggers if the request couldn't be made  
    alert(`Network Error`);  
};  
  
xhr.onprogress = function(event) { // triggers periodically  
    // event.loaded - how many bytes downloaded  
    // event.lengthComputable = true if the server sent Content-Length header  
    // event.total - total number of bytes (if lengthComputable)  
    alert(`Received ${event.loaded} of ${event.total}`);  
};
```

Here's a full example. The code below loads the URL at `/article/xmlhttprequest/example/load` from the server and prints the progress:

```
// 1. Create a new XMLHttpRequest object  
let xhr = new XMLHttpRequest();  
  
// 2. Configure it: GET-request for the URL /article/.../load  
xhr.open('GET', '/article/xmlhttprequest/example/load');  
  
// 3. Send the request over the network  
xhr.send();  
  
// 4. This will be called after the response is received  
xhr.onload = function() {  
    if (xhr.status !== 200) { // analyze HTTP status of the response  
        alert(`Error ${xhr.status}: ${xhr.statusText}`); // e.g. 404: Not Found  
    } else { // show the result  
        alert(`Done, got ${xhr.response.length} bytes`); // response is the server  
    }  
}
```

```

};

xhr.onprogress = function(event) {
  if (event.lengthComputable) {
    alert(`Received ${event.loaded} of ${event.total} bytes`);
  } else {
    alert(`Received ${event.loaded} bytes`); // no Content-Length
  }
};

xhr.onerror = function() {
  alert("Request failed");
};

```

Once the server has responded, we can receive the result in the following `xhr` properties:

status

HTTP status code (a number): `200`, `404`, `403` and so on, can be `0` in case of a non-HTTP failure.

statusText

HTTP status message (a string): usually `OK` for `200`, `Not Found` for `404`, `Forbidden` for `403` and so on.

response (old scripts may use responseText)

The server response body.

We can also specify a timeout using the corresponding property:

```

xhr.timeout = 10000; // timeout in ms, 10 seconds

```

If the request does not succeed within the given time, it gets canceled and `timeout` event triggers.

URL search parameters

To add parameters to URL, like `?name=value`, and ensure the proper encoding, we can use `URL` object:

```
let url = new URL('https://google.com/search');
url.searchParams.set('q', 'test me!');

// the parameter 'q' is encoded
xhr.open('GET', url); // https://google.com/search?q=test+me%21
```

Response Type

We can use `xhr.responseType` property to set the response format:

- `"` (default) – get as string,
- `"text"` – get as string,
- `"arraybuffer"` – get as `ArrayBuffer` (for binary data, see chapter [ArrayBuffer, binary arrays](#)),
- `"blob"` – get as `Blob` (for binary data, see chapter [Blob](#)),
- `"document"` – get as XML document (can use XPath and other XML methods) or HTML document (based on the MIME type of the received data),
- `"json"` – get as JSON (parsed automatically).

For example, let's get the response as JSON:

```
let xhr = new XMLHttpRequest();

xhr.open('GET', '/article/xmlhttprequest/example/json');

xhr.responseType = 'json';

xhr.send();

// the response is {"message": "Hello, world!"}
xhr.onload = function() {
  let responseObj = xhr.response;
  alert(responseObj.message); // Hello, world!
};
```

i Please note:

In the old scripts you may also find `xhr.responseText` and even `xhr.responseXML` properties.

They exist for historical reasons, to get either a string or XML document. Nowadays, we should set the format in `xhr.responseType` and get `xhr.response` as demonstrated above.

Ready states

`XMLHttpRequest` changes between states as it progresses. The current state is accessible as `xhr.readyState`.

All states, as in [the specification](#) ↗ :

```
UNSENT = 0; // initial state
OPENED = 1; // open called
HEADERS_RECEIVED = 2; // response headers received
LOADING = 3; // response is loading (a data packet is received)
DONE = 4; // request complete
```

An `XMLHttpRequest` object travels them in the order `0 → 1 → 2 → 3 → ... → 3 → 4`. State `3` repeats every time a data packet is received over the network.

We can track them using `readystatechange` event:

```
xhr.onreadystatechange = function() {
  if (xhr.readyState == 3) {
    // loading
  }
  if (xhr.readyState == 4) {
    // request finished
  }
};
```

You can find `readystatechange` listeners in really old code, it's there for historical reasons, as there was a time when there were no `load` and other events. Nowadays, `load/error/progress` handlers deprecate it.

Aborting request

We can terminate the request at any time. The call to `xhr.abort()` does that:

```
xhr.abort(); // terminate the request
```

That triggers `abort` event, and `xhr.status` becomes `0`.

Synchronous requests

If in the `open` method the third parameter `async` is set to `false`, the request is made synchronously.

In other words, JavaScript execution pauses at `send()` and resumes when the response is received. Somewhat like `alert` or `prompt` commands.

Here's the rewritten example, the 3rd parameter of `open` is `false`:

```
let xhr = new XMLHttpRequest();

xhr.open('GET', '/article/xmlhttprequest/hello.txt', false);

try {
  xhr.send();
  if (xhr.status !== 200) {
    alert(`Error ${xhr.status}: ${xhr.statusText}`);
  } else {
    alert(xhr.response);
  }
} catch(err) { // instead of onerror
  alert("Request failed");
}
```

It might look good, but synchronous calls are used rarely, because they block in-page JavaScript till the loading is complete. In some browsers it becomes impossible to scroll. If a synchronous call takes too much time, the browser may suggest to close the “hanging” webpage.

Many advanced capabilities of `XMLHttpRequest`, like requesting from another domain or specifying a timeout, are unavailable for synchronous requests. Also, as you can see, no progress indication.

Because of all that, synchronous requests are used very sparingly, almost never. We won't talk about them any more.

HTTP-headers

`XMLHttpRequest` allows both to send custom headers and read headers from the response.

There are 3 methods for HTTP-headers:

`setRequestHeader(name, value)`

Sets the request header with the given `name` and `value`.

For instance:

```
xhr.setRequestHeader('Content-Type', 'application/json');
```

Headers limitations

Several headers are managed exclusively by the browser, e.g. `Referer` and `Host`. The full list is [in the specification](#).

`XMLHttpRequest` is not allowed to change them, for the sake of user safety and correctness of the request.

Can't remove a header

Another peculiarity of `XMLHttpRequest` is that one can't undo `setRequestHeader`.

Once the header is set, it's set. Additional calls add information to the header, don't overwrite it.

For instance:

```
xhr.setRequestHeader('X-Auth', '123');
xhr.setRequestHeader('X-Auth', '456');

// the header will be:
// X-Auth: 123, 456
```

`getResponseHeader(name)`

Gets the response header with the given `name` (except `Set-Cookie` and `Set-Cookie2`).

For instance:

```
xhr.getResponseHeader('Content-Type')
```

`getAllResponseHeaders()`

Returns all response headers, except `Set-Cookie` and `Set-Cookie2`.

Headers are returned as a single line, e.g.:

```
Cache-Control: max-age=31536000
Content-Length: 4260
Content-Type: image/png
Date: Sat, 08 Sep 2012 16:53:16 GMT
```

The line break between headers is always `"\r\n"` (doesn't depend on OS), so we can easily split it into individual headers. The separator between the name and the value is always a colon followed by a space `": "`. That's fixed in the specification.

So, if we want to get an object with name/value pairs, we need to throw in a bit JS.

Like this (assuming that if two headers have the same name, then the latter one overwrites the former one):

```
let headers = xhr
  .getAllResponseHeaders()
  .split('\r\n')
  .reduce((result, current) => {
    let [name, value] = current.split(': ');
    result[name] = value;
    return result;
  }, {});

// headers['Content-Type'] = 'image/png'
```

POST, FormData

To make a POST request, we can use the built-in [FormData](#) object.

The syntax:

```
let formData = new FormData([form]); // creates an object, optionally fill from
formData.append(name, value); // appends a field
```

We create it, optionally fill from a form, `append` more fields if needed, and then:

1. `xhr.open('POST', ...)` – use `POST` method.
2. `xhr.send(formData)` to submit the form to the server.

For instance:

```
<form name="person">
  <input name="name" value="John">
  <input name="surname" value="Smith">
</form>
```

```

<script>
  // pre-fill FormData from the form
  let formData = new FormData(document.forms.person);

  // add one more field
  formData.append("middle", "Lee");

  // send it out
  let xhr = new XMLHttpRequest();
  xhr.open("POST", "/article/xmlhttprequest/post/user");
  xhr.send(formData);

  xhr.onload = () => alert(xhr.response);
</script>

```

The form is sent with `multipart/form-data` encoding.

Or, if we like JSON more, then `JSON.stringify` and send as a string.

Just don't forget to set the header `Content-Type: application/json`, many server-side frameworks automatically decode JSON with it:

```

let xhr = new XMLHttpRequest();

let json = JSON.stringify({
  name: "John",
  surname: "Smith"
});

xhr.open("POST", '/submit')
xhr.setRequestHeader('Content-type', 'application/json; charset=utf-8');

xhr.send(json);

```

The `.send(body)` method is pretty omnivore. It can send almost any `body`, including `Blob` and `BufferSource` objects.

Upload progress

The `progress` event triggers only on the downloading stage.

That is: if we `POST` something, `XMLHttpRequest` first uploads our data (the request body), then downloads the response.

If we're uploading something big, then we're surely more interested in tracking the upload progress. But `xhr.onprogress` doesn't help here.

There's another object, without methods, exclusively to track upload events:

`xhr.upload`.

It generates events, similar to `xhr`, but `xhr.upload` triggers them solely on uploading:

- `loadstart` – upload started.
- `progress` – triggers periodically during the upload.
- `abort` – upload aborted.
- `error` – non-HTTP error.
- `load` – upload finished successfully.
- `timeout` – upload timed out (if `timeout` property is set).
- `loadend` – upload finished with either success or error.

Example of handlers:

```
xhr.upload.onprogress = function(event) {  
    alert(`Uploaded ${event.loaded} of ${event.total} bytes`);  
};  
  
xhr.upload.onload = function() {  
    alert(`Upload finished successfully.`);  
};  
  
xhr.upload.onerror = function() {  
    alert(`Error during the upload: ${xhr.status}`);  
};
```

Here's a real-life example: file upload with progress indication:

```
<input type="file" onchange="upload(this.files[0])">  
  
<script>  
function upload(file) {  
    let xhr = new XMLHttpRequest();  
  
    // track upload progress  
    xhr.upload.onprogress = function(event) {  
        console.log(`Uploaded ${event.loaded} of ${event.total}`);  
    };  
  
    // track completion: both successful or not  
    xhr.onloadend = function() {  
        if (xhr.status == 200) {  
            console.log("success");  
        } else {  
            console.log("error " + this.status);  
        }  
    };  
}
```

```

    }
};

xhr.open("POST", "/article/xmlhttprequest/post/upload");
xhr.send(file);
}
</script>

```

Cross-origin requests

`XMLHttpRequest` can make cross-origin requests, using the same CORS policy as `fetch`.

Just like `fetch`, it doesn't send cookies and HTTP-authorization to another origin by default. To enable them, set `xhr.withCredentials` to `true`:

```

let xhr = new XMLHttpRequest();
xhr.withCredentials = true;

xhr.open('POST', 'http://anywhere.com/request');
...

```

See the chapter [Fetch: Cross-Origin Requests](#) for details about cross-origin headers.

Summary

Typical code of the GET-request with `XMLHttpRequest`:

```

let xhr = new XMLHttpRequest();

xhr.open('GET', '/my/url');

xhr.send();

xhr.onload = function() {
  if (xhr.status !== 200) { // HTTP error?
    // handle error
    alert( 'Error: ' + xhr.status);
    return;
  }

  // get the response from xhr.response
};

xhr.onprogress = function(event) {
  // report progress

```

WebSocket

The `WebSocket` protocol, described in the specification [RFC 6455](#), provides a way to exchange data between browser and server via a persistent connection. The data can be passed in both directions as “packets”, without breaking the connection and the need of additional HTTP-requests.

WebSocket is especially great for services that require continuous data exchange, e.g. online games, real-time trading systems and so on.

A simple example

To open a websocket connection, we need to create `new WebSocket` using the special protocol `ws` in the url:

```
let socket = new WebSocket("ws://javascript.info");
```

There's also encrypted `wss://` protocol. It's like HTTPS for websockets.

i Always prefer `wss://`

The `wss://` protocol is not only encrypted, but also more reliable.

That's because `ws://` data is not encrypted, visible for any intermediary. Old proxy servers do not know about WebSocket, they may see "strange" headers and abort the connection.

On the other hand, `wss://` is WebSocket over TLS, (same as HTTPS is HTTP over TLS), the transport security layer encrypts the data at the sender and decrypts it at the receiver. So data packets are passed encrypted through proxies. They can't see what's inside and let them through.

Once the socket is created, we should listen to events on it. There are totally 4 events:

- **open** – connection established,
- **message** – data received,
- **error** – websocket error,
- **close** – connection closed.

...And if we'd like to send something, then `socket.send(data)` will do that.

Here's an example:

```
let socket = new WebSocket("wss://javascript.info/article/websocket/demo/hello");

socket.onopen = function(e) {
    alert("[open] Connection established");
    alert("Sending to server");
    socket.send("My name is John");
};

socket.onmessage = function(event) {
    alert(`[message] Data received from server: ${event.data}`);
};

socket.onclose = function(event) {
    if (event.wasClean) {
        alert(`[close] Connection closed cleanly, code=${event.code} reason=${event.reason}`);
    } else {
        // e.g. server process killed or network down
        // event.code is usually 1006 in this case
        alert(`[close] Connection died`);
    }
};

socket.onerror = function(error) {
```

```
    alert(`[error]`);  
};
```

For demo purposes, there's a small server [server.js](#) written in Node.js, for the example above, running. It responds with “Hello from server, John”, then waits 5 seconds and closes the connection.

So you'll see events `open` → `message` → `close`.

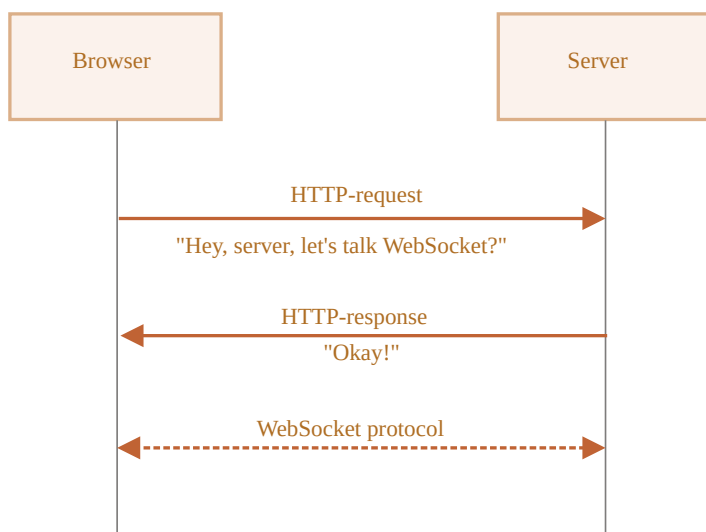
That's actually it, we can talk WebSocket already. Quite simple, isn't it?

Now let's talk more in-depth.

Opening a websocket

When `new WebSocket(url)` is created, it starts connecting immediately.

During the connection, the browser (using headers) asks the server: “Do you support WebSocket?” And if the server replies “yes”, then the talk continues in WebSocket protocol, which is not HTTP at all.



Here's an example of browser headers for a request made by `new WebSocket("wss://javascript.info/chat")`.

```
GET /chat  
Host: javascript.info  
Origin: https://javascript.info  
Connection: Upgrade  
Upgrade: websocket  
Sec-WebSocket-Key: Iv8io/9s+lYFgZWcXczP8Q==  
Sec-WebSocket-Version: 13
```

- **Origin** – the origin of the client page, e.g. `https://javascript.info`. WebSocket objects are cross-origin by nature. There are no special headers or other limitations. Old servers are unable to handle WebSocket anyway, so there are no compatibility issues. But the **Origin** header is important, as it allows the server to decide whether or not to talk WebSocket with this website.
- **Connection: Upgrade** – signals that the client would like to change the protocol.
- **Upgrade: websocket** – the requested protocol is “websocket”.
- **Sec-WebSocket-Key** – a random browser-generated key, used to ensure that the server supports WebSocket protocol. It's random to prevent proxies from caching any following communication.
- **Sec-WebSocket-Version** – WebSocket protocol version, 13 is the current one.

i WebSocket handshake can't be emulated

We can't use `XMLHttpRequest` or `fetch` to make this kind of HTTP-request, because JavaScript is not allowed to set these headers.

If the server agrees to switch to WebSocket, it should send code 101 response:

```
101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: hsBlbuDTkk24srzE0TBULZAlC2g=
```

Here **Sec-WebSocket-Accept** is **Sec-WebSocket-Key**, recoded using a special algorithm. Upon seeing it, the browser understands that the server really does support the WebSocket protocol.

Afterwards, the data is transferred using the WebSocket protocol, we'll see its structure (“frames”) soon. And that's not HTTP at all.

Extensions and subprotocols

There may be additional headers **Sec-WebSocket-Extensions** and **Sec-WebSocket-Protocol** that describe extensions and subprotocols.

For instance:

- **Sec-WebSocket-Extensions: deflate-frame** means that the browser supports data compression. An extension is something related to transferring the data, functionality that extends the WebSocket protocol. The header **Sec-WebSocket-Extensions** is sent automatically by the browser, with the list of all extensions it supports.

- `Sec-WebSocket-Protocol: soap, wamp` means that we'd like to transfer not just any data, but the data in [SOAP](#) or WAMP ("The WebSocket Application Messaging Protocol") protocols. WebSocket subprotocols are registered in the [IANA catalogue](#). So, this header describes the data formats that we're going to use.

This optional header is set using the second parameter of `new WebSocket`. That's the array of subprotocols, e.g. if we'd like to use SOAP or WAMP:

```
let socket = new WebSocket("wss://javascript.info/chat", ["soap", "wamp"]);
```

The server should respond with a list of protocols and extensions that it agrees to use.

For example, the request:

```
GET /chat
Host: javascript.info
Upgrade: websocket
Connection: Upgrade
Origin: https://javascript.info
Sec-WebSocket-Key: Iv8io/9s+lYFgZWcXczP8Q==
Sec-WebSocket-Version: 13
Sec-WebSocket-Extensions: deflate-frame
Sec-WebSocket-Protocol: soap, wamp
```

Response:

```
101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: hsBlbuDTkk24srzE0TBULZA1C2g=
Sec-WebSocket-Extensions: deflate-frame
Sec-WebSocket-Protocol: soap
```

Here the server responds that it supports the extension "deflate-frame", and only SOAP of the requested subprotocols.

Data transfer

WebSocket communication consists of "frames" – data fragments, that can be sent from either side, and can be of several kinds:

- "text frames" – contain text data that parties send to each other.

- “binary data frames” – contain binary data that parties send to each other.
- “ping/pong frames” are used to check the connection, sent from the server, the browser responds to these automatically.
- there’s also “connection close frame” and a few other service frames.

In the browser, we directly work only with text or binary frames.

WebSocket .send() method can send either text or binary data.

A call `socket.send(body)` allows `body` in string or a binary format, including `Blob`, `ArrayBuffer`, etc. No settings are required: just send it out in any format.

When we receive the data, text always comes as string. And for binary data, we can choose between `Blob` and `ArrayBuffer` formats.

That’s set by `socket.binaryType` property, it’s `"blob"` by default, so binary data comes as `Blob` objects.

`Blob` is a high-level binary object, it directly integrates with `<a>`, `<img>` and other tags, so that’s a sane default. But for binary processing, to access individual data bytes, we can change it to `"arraybuffer"`:

```
socket.binaryType = "arraybuffer";
socket.onmessage = (event) => {
  // event.data is either a string (if text) or arraybuffer (if binary)
};
```

Rate limiting

Imagine, our app is generating a lot of data to send. But the user has a slow network connection, maybe on a mobile internet, outside of a city.

We can call `socket.send(data)` again and again. But the data will be buffered (stored) in memory and sent out only as fast as network speed allows.

The `socket.bufferedAmount` property stores how many bytes remain buffered at this moment, waiting to be sent over the network.

We can examine it to see whether the socket is actually available for transmission.

```
// every 100ms examine the socket and send more data
// only if all the existing data was sent out
setInterval(() => {
  if (socket.bufferedAmount == 0) {
    socket.send(moreData());
  }
}, 100);
```

Connection close

Normally, when a party wants to close the connection (both browser and server have equal rights), they send a “connection close frame” with a numeric code and a textual reason.

The method for that is:

```
socket.close([code], [reason]);
```

- `code` is a special WebSocket closing code (optional)
- `reason` is a string that describes the reason of closing (optional)

Then the other party in the `close` event handler gets the code and the reason, e.g.:

```
// closing party:
socket.close(1000, "Work complete");

// the other party
socket.onclose = event => {
  // event.code === 1000
  // event.reason === "Work complete"
  // event.wasClean === true (clean close)
};
```

Most common code values:

- `1000` – the default, normal closure (used if no `code` supplied),
- `1006` – no way to set such code manually, indicates that the connection was lost (no close frame).

There are other codes like:

- `1001` – the party is going away, e.g. server is shutting down, or a browser leaves the page,
- `1009` – the message is too big to process,
- `1011` – unexpected error on server,
- ...and so on.

The full list can be found in [RFC6455, §7.4.1](#).

WebSocket codes are somewhat like HTTP codes, but different. In particular, codes lower than `1000` are reserved, there'll be an error if we try to set such a code.

```
// in case connection is broken
socket.onclose = event => {
  // event.code === 1006
  // event.reason === ""
  // event.wasClean === false (no closing frame)
};
```

Connection state

To get connection state, additionally there's `socket.readyState` property with values:

- **0** – “CONNECTING”: the connection has not yet been established,
- **1** – “OPEN”: communicating,
- **2** – “CLOSING”: the connection is closing,
- **3** – “CLOSED”: the connection is closed.

Chat example

Let's review a chat example using browser WebSocket API and Node.js WebSocket module <https://github.com/websockets/ws> . We'll pay the main attention to the client side, but the server is also simple.

HTML: we need a `<form>` to send messages and a `<div>` for incoming messages:

```
<!-- message form -->
<form name="publish">
  <input type="text" name="message">
  <input type="submit" value="Send">
</form>

<!-- div with messages -->
<div id="messages"></div>
```

From JavaScript we want three things:

1. Open the connection.
2. On form submission – `socket.send(message)` for the message.
3. On incoming message – append it to `div#messages` .

Here's the code:

```

let socket = new WebSocket("wss://javascript.info/article/websocket/chat/ws");

// send message from the form
document.forms.publish.onsubmit = function() {
  let outgoingMessage = this.message.value;

  socket.send(outgoingMessage);
  return false;
};

// message received - show the message in div#messages
socket.onmessage = function(event) {
  let message = event.data;

  let messageElem = document.createElement('div');
  messageElem.textContent = message;
  document.getElementById('messages').prepend(messageElem);
}

```

Server-side code is a little bit beyond our scope. Here we'll use Node.js, but you don't have to. Other platforms also have their means to work with WebSocket.

The server-side algorithm will be:

1. Create `clients = new Set()` – a set of sockets.
2. For each accepted websocket, add it to the set `clients.add(socket)` and set `message` event listener to get its messages.
3. When a message is received: iterate over clients and send it to everyone.
4. When a connection is closed: `clients.delete(socket)`.

```

const ws = new require('ws');
const wss = new ws.Server({noServer: true});

const clients = new Set();

http.createServer((req, res) => {
  // here we only handle websocket connections
  // in real project we'd have some other code here to handle non-websocket req
  wss.handleUpgrade(req, req.socket, Buffer.alloc(0), onSocketConnect);
});

function onSocketConnect(ws) {
  clients.add(ws);

  ws.on('message', function(message) {
    message = message.slice(0, 50); // max message length will be 50

    for(let client of clients) {
      client.send(message);
    }
  });
}

```

```
    }  
  });  
  
  ws.on('close', function() {  
    clients.delete(ws);  
  });  
}
```

Here's the working example:

Send

You can also download it (upper-right button in the iframe) and run it locally. Just don't forget to install [Node.js](#) and `npm install ws` before running.

Summary

WebSocket is a modern way to have persistent browser-server connections.

- WebSockets don't have cross-origin limitations.
- They are well-supported in browsers.
- Can send/receive strings and binary data.

The API is simple.

Methods:

- `socket.send(data)`,
- `socket.close([code], [reason])`.

Events:

- `open`,
- `message`,
- `error`,
- `close`.

WebSocket by itself does not include reconnection, authentication and many other high-level mechanisms. So there are client/server libraries for that, and it's also possible to implement these capabilities manually.

Sometimes, to integrate WebSocket into existing projects, people run a WebSocket server in parallel with the main HTTP-server, and they share a single database. Requests to WebSocket use `wss://ws.site.com`, a subdomain that leads to the WebSocket server, while `https://site.com` goes to the main HTTP-server.

Surely, other ways of integration are also possible.

Server Sent Events

The [Server-Sent Events](#) specification describes a built-in class `EventSource`, that keeps connection with the server and allows to receive events from it.

Similar to `WebSocket`, the connection is persistent.

But there are several important differences:

WebSocket	EventSource
Bi-directional: both client and server can exchange messages	One-directional: only server sends data
Binary and text data	Only text
WebSocket protocol	Regular HTTP

`EventSource` is a less-powerful way of communicating with the server than `WebSocket`.

Why should one ever use it?

The main reason: it's simpler. In many applications, the power of `WebSocket` is a little bit too much.

We need to receive a stream of data from server: maybe chat messages or market prices, or whatever. That's what `EventSource` is good at. Also it supports auto-reconnect, something we need to implement manually with `WebSocket`. Besides, it's a plain old HTTP, not a new protocol.

Getting messages

To start receiving messages, we just need to create `new EventSource(url)`.

The browser will connect to `url` and keep the connection open, waiting for events.

The server should respond with status 200 and the header `Content-Type: text/event-stream`, then keep the connection and write messages into it in the special format, like this:

```
data: Message 1

data: Message 2

data: Message 3
data: of two lines
```

- A message text goes after `data:` , the space after the colon is optional.
- Messages are delimited with double line breaks `\n\n`.
- To send a line break `\n` , we can immediately send one more `data:` (3rd message above).

In practice, complex messages are usually sent JSON-encoded. Line-breaks are encoded as `\n` within them, so multiline `data:` messages are not necessary.

For instance:

```
data: {"user":"John","message":"First line\n Second line"}
```

...So we can assume that one `data:` holds exactly one message.

For each such message, the `message` event is generated:

```
let eventSource = new EventSource("/events/subscribe");

eventSource.onmessage = function(event) {
  console.log("New message", event.data);
  // will log 3 times for the data stream above
};

// or eventSource.addEventListener('message', ...)
```

Cross-origin requests

`EventSource` supports cross-origin requests, like `fetch` and any other networking methods. We can use any URL:

```
let source = new EventSource("https://another-site.com/events");
```

The remote server will get the `Origin` header and must respond with `Access-Control-Allow-Origin` to proceed.

To pass credentials, we should set the additional option `withCredentials` , like this:

```
let source = new EventSource("https://another-site.com/events", {
  withCredentials: true
});
```

Please see the chapter [Fetch: Cross-Origin Requests](#) for more details about cross-origin headers.

Reconnection

Upon creation, `new EventSource` connects to the server, and if the connection is broken – reconnects.

That's very convenient, as we don't have to care about it.

There's a small delay between reconnections, a few seconds by default.

The server can set the recommended delay using `retry:` in response (in milliseconds):

```
retry: 15000
data: Hello, I set the reconnection delay to 15 seconds
```

The `retry:` may come both together with some data, or as a standalone message.

The browser should wait that many milliseconds before reconnecting. Or longer, e.g. if the browser knows (from OS) that there's no network connection at the moment, it may wait until the connection appears, and then retry.

- If the server wants the browser to stop reconnecting, it should respond with HTTP status 204.
- If the browser wants to close the connection, it should call `eventSource.close()`:

```
let eventSource = new EventSource(...);

eventSource.close();
```

Also, there will be no reconnection if the response has an incorrect `Content-Type` or its HTTP status differs from 301, 307, 200 and 204. In such cases the `"error"` event will be emitted, and the browser won't reconnect.

Please note:

When a connection is finally closed, there's no way to "reopen" it. If we'd like to connect again, just create a new `EventSource`.

Message id

When a connection breaks due to network problems, either side can't be sure which messages were received, and which weren't.

To correctly resume the connection, each message should have an `id` field, like this:

```
data: Message 1
id: 1

data: Message 2
id: 2

data: Message 3
data: of two lines
id: 3
```

When a message with `id` is received, the browser:

- Sets the property `eventSource.lastEventId` to its value.
- Upon reconnection sends the header `Last-Event-ID` with that `id`, so that the server may re-send following messages.

i Put `id` after `data`:

Please note: the `id` is appended below message `data` by the server, to ensure that `lastEventId` is updated after the message is received.

Connection status: `readyState`

The `EventSource` object has `readyState` property, that has one of three values:

```
EventSource.CONNECTING = 0; // connecting or reconnecting
EventSource.OPEN = 1;      // connected
EventSource.CLOSED = 2;    // connection closed
```

When an object is created, or the connection is down, it's always `EventSource.CONNECTING` (equals `0`).

We can query this property to know the state of `EventSource`.

- `error` – the connection could not be established, e.g. the server returned HTTP 500 status.

The server may specify another type of event with `event: ...` at the event start.

For example:

```
event: join
data: Bob

data: Hello

event: leave
data: Bob
```

To handle custom events, we must use `addEventListener`, not `onmessage`:

```
eventSource.addEventListener('join', event => {
  alert(`Joined ${event.data}`);
});

eventSource.addEventListener('message', event => {
  alert(`Said: ${event.data}`);
});

eventSource.addEventListener('leave', event => {
  alert(`Left ${event.data}`);
});
```

Full example

Here's the server that sends messages with `1`, `2`, `3`, then `bye` and breaks the connection.

Then the browser automatically reconnects.

<https://plnkr.co/edit/etHvCsT69QRervwf?p=preview> ↗

Summary

`EventSource` object automatically establishes a persistent connection and allows the server to send messages over it.

It offers:

- Automatic reconnect, with tunable `retry` timeout.